Developer Manual - COMP3211 PIM

Group: 89. Member names and student IDs await human input. **Revision:** 2026-10-23. **Platform:** Windows 11, PowerShell. **Verified interpreters:** Python 3.11.9, 3.12.3 and 3.13.5. **Delivery version:** Independent review candidate, 2026-10-23. Use the accompanying manifest to identify the delivered source, tests and documents by their actual file hashes; an earlier source commit does not identify this complete delivery.

1. Open and run the delivered project

Extract the ZIP into a folder of your choice. Open PowerShell **inside its `03_Implementation` folder**, which contains `src`, `tests`, `tools` and this manual. No repository access or Git metadata is needed. The following commands are relative to that folder:

```
Get-ChildItem src, tests, tools
python --version
python src/main.py
```

Select one of the verified Python versions. No installation of third-party Python packages, compilation step or build command is required. The project uses the standard library only. `python src/main.py` opens the interactive terminal prompt `pim>`; it does not open a separate application window.

At the prompt, enter:

```
help
add note "memo"
show 1
save "records.pim"
exit
```

Launch `python src/main.py` again from the same folder, then enter `load "records.pim"`, `show all` and `exit`. A new process starts with an empty collection. Saving is explicit; exiting does not automatically save. The User Manual explains commands, formats and errors.

2. Test and measure coverage

From the same source-code root:

```
$env:PYTHONPATH = "$PWD\src"
python -m unittest discover -s tests -t . -v
python tools/model_coverage.py
python -m compileall -q src tests tools
```

The explicit `-t .` prevents `tests/model` from shadowing `src/model`. The suite includes model unit tests, controller and persistence tests, real CLI subprocesses, and coverage-tool regression tests. The current test count, interpreter-specific results and evidence files are recorded in `Test_Coverage.pdf`. If multiple registered versions are installed, substitute an appropriate launcher such as `py -3.12` for `python`. The coverage tool reports **model source-line coverage**, excludes non-source `0`/`None` bookkeeping entries, and lists unexecuted source lines. It does not measure branch coverage or subprocess execution. Use the interpreter named in `Test_Coverage.pdf` to reproduce its exact count. `compileall` exits with code 0 on success and creates bytecode caches, which are excluded from the submission.

The runtime used to verify Python 3.11.9 was a standalone official Windows embeddable distribution, configured for normal project imports. It is not included in the submission. You may use a normal installed Python interpreter matching the verified version. These results establish the tested versions, not all future Python versions.

3. IDE and component map

Visual Studio Code with a Python extension is a suitable editor. Open `03_Implementation`, select a verified interpreter, and run the commands above in its integrated PowerShell terminal. No IDE version is required; recorded results came from terminal commands.

Path	Responsibility
`src/main.py`	Creates the manager/controller and starts the terminal session.

Path	Responsibility
`src/model/records.py`, `validation.py`, `errors.py`	Immutable record types, value checks and domain errors.
`src/model/manager.py`	Collection, stable IDs, create/list/get/update/delete/search and snapshot reconstruction.
`src/model/search.py`	Criterion nodes, search tokenization, parsing and evaluation.
`src/controller/cli.py`	Command parsing, output, error recovery and coordination.
`src/storage/pim_file.py`	Validated UTF-8 JSON `.pim` save/load and atomic file replacement.
`tests/model/`, `tests/controller/`, `tests/storage/`	Direct model and boundary tests, including targeted requirement checks.
`tests/test_acceptance.py`	Actual CLI subprocess acceptance tests.
`tools/model_coverage.py`, `tests/tools/`	Standard-library trace runner for model source-line coverage and its measurement regression tests.

4. Debugging and troubleshooting

To debug using only the standard library, run:

```
python -m pdb src/main.py
```

At `(Pdb)`, enter `b main`, then `c` to stop at `main()`, and `c` again to reach `pim>`. Enter application commands normally. After `exit`, pdb offers to restart; enter `q` at `(Pdb)` to leave the debugger. These exact steps were exercised during the independent review.

For IDE debugging, set the debug program to `src/main.py`, working directory to `03_Implementation`, and console to the integrated terminal. No program arguments are needed. For direct test debugging, set `PYTHONPATH` to the `src` folder. The normal program launch already places that directory on the import path.

Useful breakpoints are `CommandController._dispatch`, `PIMManager.update`, `parse_criterion` and `load_pim`. Expected invalid input is printed as an error and the program accepts another command. For a failed load, inspect validation before the controller replaces its manager. For a failed update, inspect replacement-record validation before collection assignment.

If Python cannot find `src/main.py`, check the current directory. If tests cannot import `model`, check both `PYTHONPATH` and `-t .`. If a saved file cannot be found, check the process working directory and exact lowercase `.pim` suffix. Do not run `pip install` to resolve these path errors: the delivered product has no third-party dependency.